

FinalSourceCodeNHSCode

January 6, 2025

```
[1]: # Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier,
    ↪ GradientBoostingClassifier, AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, roc_auc_score,
    ↪ confusion_matrix, ConfusionMatrixDisplay
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import GridSearchCV
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Load dataset
data = pd.read_csv("DataSet.csv")
```

```
[2]: # Rename columns for consistency
data.rename(columns={
    'Asian/Asian British': 'Asian',
    'Black/African/Caribbean/Black British': 'Black',
    'Mixed/Multiple ethnic groups': 'Mixed',
    'Not Stated': 'Not_Stated',
    'Other ethnic group': 'Other',
    'Clinical condition group': 'Condition_Group'
}, inplace=True)
```

```
[3]: # Display summary statistics
print("Summary Statistics:")
print(data.describe())
```

```
# Check for missing values
print("\nMissing Values:")
print(data.isnull().sum())
```

Summary Statistics:

	Total	Female	Male	12 to 16	17 to 19 \
count	1.100000e+01	11.000000	11.000000	11.000000	11.000000
mean	3.540621e+05	189601.818182	164432.181818	3796.181818	2625.000000
std	4.507134e+05	242088.497897	209385.217254	4959.784730	3445.109606
min	3.031400e+04	16371.000000	11944.000000	139.000000	116.000000
25%	1.081325e+05	47079.000000	48375.500000	1125.500000	692.000000
50%	1.684130e+05	97042.000000	79927.000000	2497.000000	1626.000000
75%	3.697205e+05	209649.500000	161763.500000	3866.500000	2674.000000
max	1.488765e+06	792553.000000	696074.000000	17546.000000	11837.000000

	20 to 29	30 to 39	40 to 49	50 to 59 \
count	11.000000	11.000000	11.000000	11.000000
mean	12264.272727	23377.000000	34407.363636	58091.181818
std	16411.590502	31427.815075	45571.378518	73819.003548
min	674.000000	2691.000000	5413.000000	4853.000000
25%	3493.500000	6308.500000	10057.500000	21350.000000
50%	5375.000000	8278.000000	13629.000000	28527.000000
75%	12089.000000	22094.500000	32167.500000	55286.500000
max	54680.000000	104669.000000	152548.000000	248465.000000

	60 to 69	70 to 79	80 to 89	90+ \
count	11.000000	11.000000	11.000000	11.000000
mean	71789.272727	85182.454545	51583.000000	10946.363636
std	88542.314442	107507.496856	68000.763477	15399.099300
min	2283.000000	433.000000	82.000000	10.000000
25%	27014.500000	21066.500000	7824.500000	1171.500000
50%	32067.000000	49062.000000	20307.000000	1959.000000
75%	79438.500000	103266.500000	71680.500000	18539.000000
max	293091.000000	342752.000000	214291.000000	48886.000000

	Asian	Black	Mixed	Not_Stated	Other \
count	11.000000	11.000000	11.000000	11.000000	11.000000
mean	19447.909091	17169.000000	4599.545455	13496.818182	4824.363636
std	24103.813169	24643.057201	6158.503574	17402.163146	6235.943125
min	1937.000000	1231.000000	658.000000	1115.000000	585.000000
25%	5462.000000	4161.500000	1673.000000	3712.500000	1599.000000
50%	8811.000000	6655.000000	2059.000000	6382.000000	2676.000000
75%	22676.500000	15077.000000	3659.500000	14235.500000	3901.000000
max	82246.000000	81961.000000	20808.000000	56879.000000	21311.000000

	White
count	1.100000e+01
mean	2.945245e+05

```
std      3.742492e+05
min      4.182000e+03
25%      8.804550e+04
50%      1.465460e+05
75%      3.201000e+05
max      1.225560e+06
```

Missing Values:

```
Condition_Group    0
Total              0
Female             0
Male              0
12 to 16           0
17 to 19           0
20 to 29           0
30 to 39           0
40 to 49           0
50 to 59           0
60 to 69           0
70 to 79           0
80 to 89           0
90+                0
Asian              0
Black              0
Mixed              0
Not_Stated         0
Other              0
White              0
dtype: int64
```

```
[4]: # Select numerical features
numerical_features = data.select_dtypes(include=['int64', 'float64']).columns

# Compute summary statistics for numerical features
summary_stats = data[numerical_features].describe()

# Plot distributions of numerical features
plt.figure(figsize=(12, 8))
for i, col in enumerate(numerical_features, 1):
    plt.subplot(int(np.ceil(len(numerical_features)/4)), 4, i)
    sns.histplot(data[col], kde=True, bins=20, color="skyblue")
    plt.title(f"{col}")
plt.tight_layout()
plt.savefig("numerical_distributions.png")
plt.show()

# Plot correlation heatmap
```

```

plt.figure(figsize=(12, 8))
sns.heatmap(data[numerical_features].corr(), annot=True, fmt=".2f")
plt.title("Correlation Heatmap")
plt.savefig("correlation_heatmap.png")
plt.show()

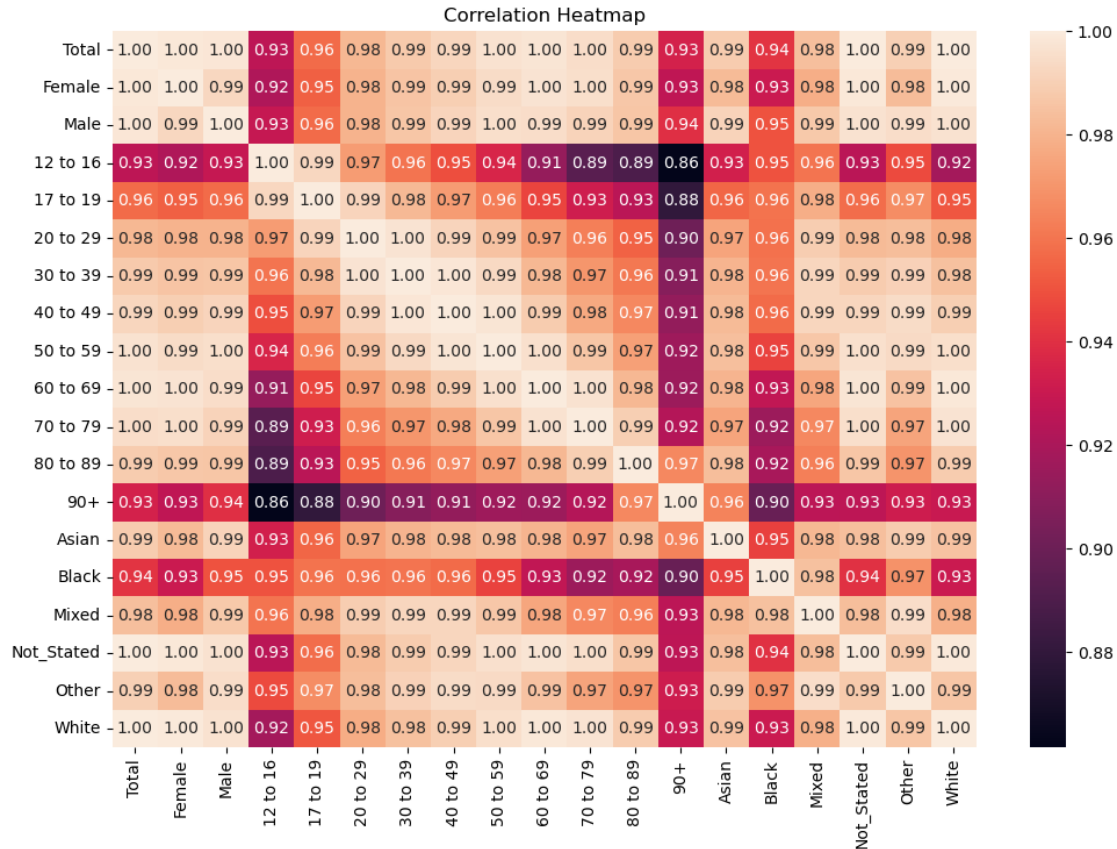
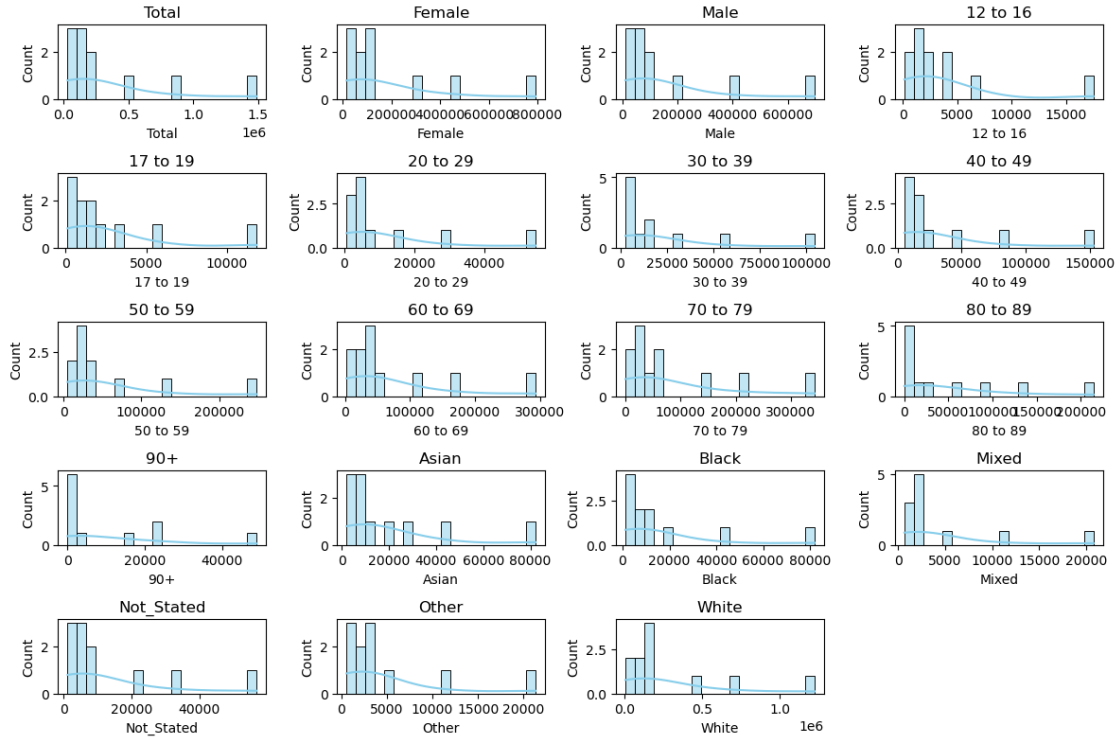
# Prepare summary statistics for plotting
stats_to_plot = ['count', 'mean', 'std', 'min', '25%', '50%', '75%', 'max']
summary_stats_to_plot = summary_stats.loc[stats_to_plot].transpose()

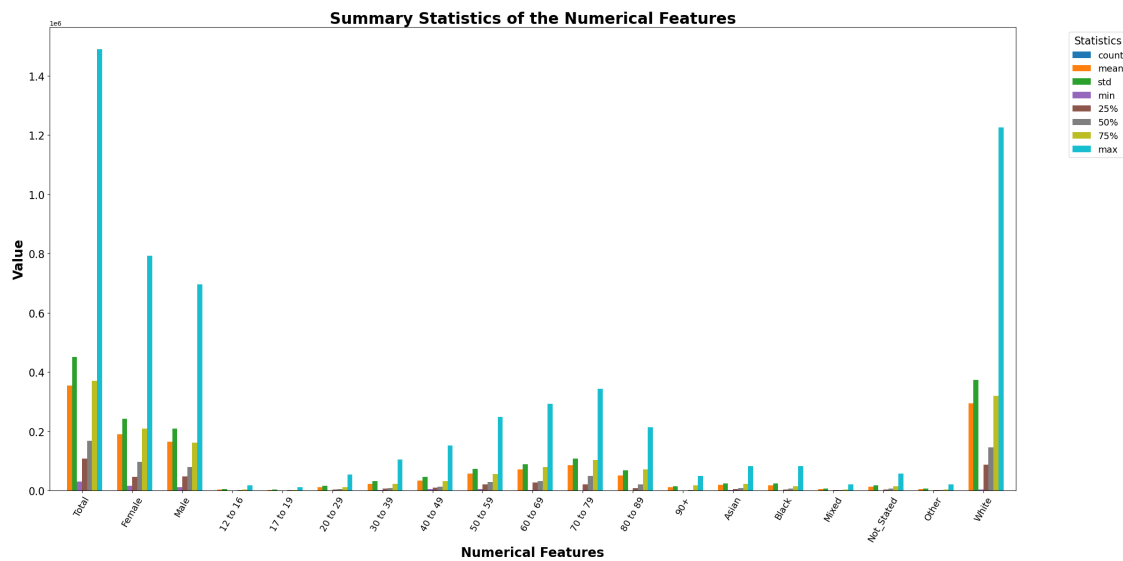
# Create a bar plot for summary statistics
fig, ax = plt.subplots(figsize=(24, 12))
summary_stats_to_plot.plot(kind='bar', width=0.8, colormap='tab10', ax=ax)

# Customize plot aesthetics
ax.set_title("Summary Statistics of the Numerical Features", fontsize=24,
    ↪weight='bold')
ax.set_xlabel("Numerical Features", fontsize=20, weight='bold')
ax.set_ylabel("Value", fontsize=20, weight='bold')
ax.tick_params(axis='x', labelrotation=60, labelsiz=14)
ax.tick_params(axis='y', labelsiz=16)
ax.legend(title="Statistics", bbox_to_anchor=(1.05, 1), loc='upper left',
    ↪fontsize=14, title_fontsize=16)

# Adjust layout and save the plot
plt.tight_layout()
plt.savefig("summary_statistics_plot_fixed.png", dpi=300, bbox_inches='tight')
plt.show()

```





```
[5]: # Rename columns for clarity and consistency
data.rename(columns={
    'Asian/Asian British': 'Asian',
    'Black/African/Caribbean/Black British': 'Black',
    'Mixed/Multiple ethnic groups': 'Mixed',
    'Not Stated': 'Not_Statd',
    'Other ethnic group': 'Other',
    'Clinical condition group': 'Condition_Group'
}, inplace=True)

# Create binary target variable for classification
data['High_Risk'] = (data['Total'] > data['Total'].median()).astype(int)

# Feature Engineering
X_features = data.drop(columns=['Condition_Group', 'Total', 'High_Risk',
    ↪ 'Female', 'Male',
    ↪ '12 to 16', '17 to 19', '20 to 29', '30 to 39',
    ↪ '40 to 49',
    ↪ '50 to 59', '60 to 69', '70 to 79', '80 to 89',
    ↪ '90+'])
X = X_features
y = data['High_Risk']
```

```
[6]: # Handle class imbalance using SMOTE
smote = SMOTE(random_state=42, k_neighbors=1)
X_balanced, y_balanced = smote.fit_resample(X, y)
```

```
[7]: # Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_balanced, y_balanced,
↳test_size=0.2, random_state=42)
```

```
[8]: # Define models for classification
classification_models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "AdaBoost": AdaBoostClassifier(random_state=42),
    "SVM": SVC(probability=True, random_state=42)
}
```

```
[9]: # Model evaluation implementation
classification_results = []
confusion_matrices = {}

for name, model in classification_models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[: , 1]

    report = classification_report(y_test, y_pred, output_dict=True)
    auc_roc = roc_auc_score(y_test, y_prob)

    classification_results.append({
        "Model": name,
        "Accuracy": report["accuracy"],
        "Precision": report["1"]["precision"],
        "Recall": report["1"]["recall"],
        "F1-Score": report["1"]["f1-score"],
        "AUC-ROC": auc_roc
    })

    confusion_matrices[name] = confusion_matrix(y_test, y_pred)
```

```
/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, msg_start, len(result))
/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, msg_start, len(result))
```

```

/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, msg_start, len(result))

```

```

[10]: # Define parameter grids for hyperparameter tuning
param_grids = {
    "Random Forest": { # Parameter grid for Random Forest
        'n_estimators': [100, 200], # Number of trees in the forest
        'max_depth': [10, 20] # Maximum depth of each tree
    },
    "Gradient Boosting": { # Parameter grid for Gradient Boosting
        'n_estimators': [100, 200], # Number of boosting stages
        'learning_rate': [0.01, 0.1] # Step size for updating weights
    }
}

# Iterate through the models and their parameter grids
for name, model in classification_models.items():
    if name in param_grids: # Check if the model has a defined parameter grid
        # Initialize GridSearchCV with model, parameter grid, and 5-fold
        ↪cross-validation
        grid_search = GridSearchCV(model, param_grids[name], cv=5)

        # Perform hyperparameter tuning by fitting the model to the training
        ↪data
        grid_search.fit(X_train, y_train)

        # Retrieve and print the best hyperparameters for the model
        print(f"Best parameters for {name}: {grid_search.best_params_}")
        print(f"Best cross-validated score for {name}: {grid_search.
        ↪best_score_}")

```

```

/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-
packages/sklearn/model_selection/_split.py:700: UserWarning: The least populated
class in y has only 4 members, which is less than n_splits=5.
warnings.warn(

```

```

Best parameters for Random Forest: {'max_depth': 10, 'n_estimators': 100}
Best cross-validated score for Random Forest: 0.9

```

```

/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-
packages/sklearn/model_selection/_split.py:700: UserWarning: The least populated
class in y has only 4 members, which is less than n_splits=5.
warnings.warn(

```

```

Best parameters for Gradient Boosting: {'learning_rate': 0.01, 'n_estimators':
100}

```


Best cross-validated score for Gradient Boosting: 0.9

```
[11]: # Model evaluation implementation

classification_results = [] # Initialize a list to store evaluation results
    ↪for each model
confusion_matrices = {} # Initialize a dictionary to store confusion
    ↪matrices for each model

for name, model in classification_models.items(): # Iterate through the
    ↪classification models
    # Train the model using the training data
    model.fit(X_train, y_train)

    # Generate predictions on the test data
    y_pred = model.predict(X_test) # Predicted class labels
    y_prob = model.predict_proba(X_test)[ :, 1] # Predicted probabilities for
    ↪the positive class (1)

    # Generate a classification report (includes metrics like precision,
    ↪recall, F1-score, accuracy)
    report = classification_report(y_test, y_pred, output_dict=True)

    # Calculate the Area Under the Receiver Operating Characteristic Curve
    ↪(AUC-ROC)
    auc_roc = roc_auc_score(y_test, y_prob)

    # Append the evaluation metrics for the current model to the results list
    classification_results.append({
        "Model": name, # Model name
        "Accuracy": report["accuracy"], # Overall accuracy
        "Precision": report["1"]["precision"], # Precision for the positive
    ↪class
        "Recall": report["1"]["recall"], # Recall (sensitivity) for the
    ↪positive class
        "F1-Score": report["1"]["f1-score"], # F1-score for the positive class
        "AUC-ROC": auc_roc # AUC-ROC score
    })

    # Compute and store the confusion matrix for the current model
    confusion_matrices[name] = confusion_matrix(y_test, y_pred)
```

```
/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, msg_start, len(result))
```

```

/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, msg_start, len(result))
/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, msg_start, len(result))

```

```

[12]: # Save and display results
classification_results_df = pd.DataFrame(classification_results)
classification_results_df.to_csv("classification_results.csv", index=False)
print(classification_results_df.columns)

# Plot performance metrics
classification_results_df.set_index("Model")[["Accuracy", "Precision",
↵ "Recall", "F1-Score"]].plot(kind="bar", figsize=(12, 8))
plt.title("Classification Metrics Comparison")
plt.ylabel("Score")
plt.xlabel("Models")
plt.legend(loc="lower right")
plt.grid(axis="y")
plt.tight_layout()
plt.savefig("classification_metrics_comparison.png")
plt.show()

# Plot confusion matrices
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for idx, (name, cm) in enumerate(confusion_matrices.items()):
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Class_
↵ 0", "Class 1"])
    disp.plot(ax=axes[idx], colorbar=False)
    axes[idx].set_title(f"Confusion Matrix ({name})")

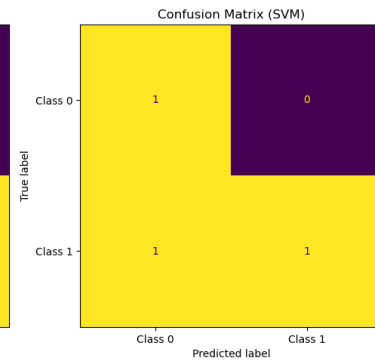
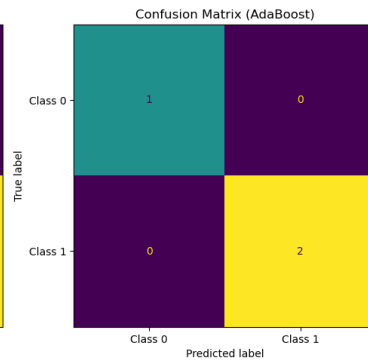
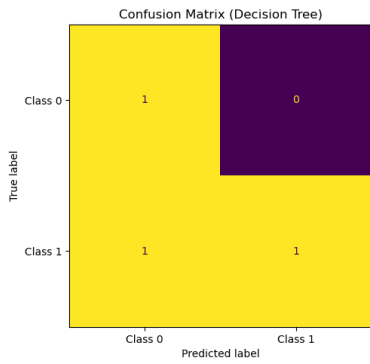
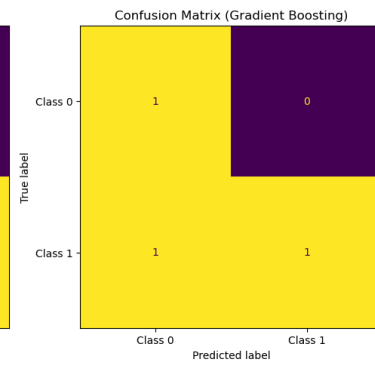
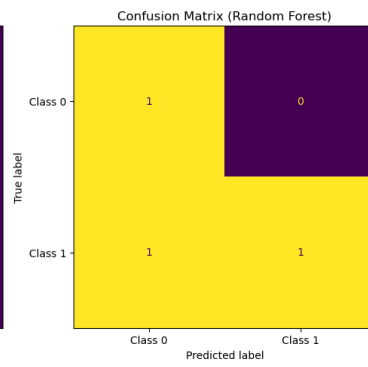
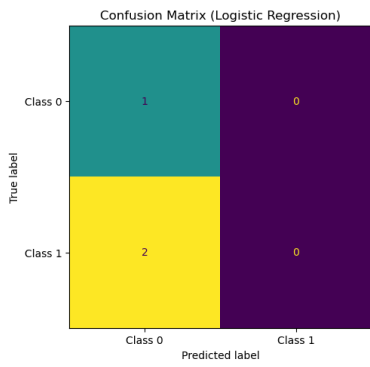
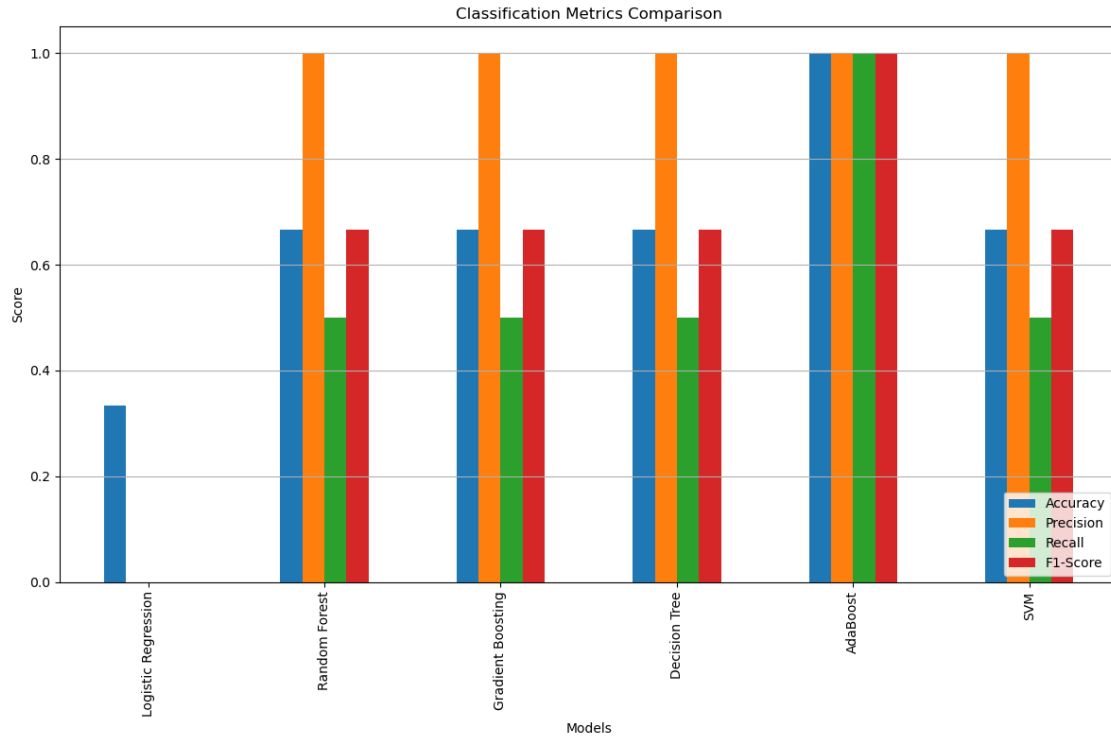
plt.tight_layout()
plt.savefig("classification_confusion_matrices.png")
plt.show()

```

```

Index(['Model', 'Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC-ROC'],
dtype='object')

```



```

[13]: import numpy as np
from sklearn.metrics import precision_score, recall_score, f1_score

# Calculate metrics for Train and Test datasets
results = []

for name, model in classification_models.items():
    try:
        # Train predictions
        y_train_pred = model.predict(X_train)
        # Test predictions
        y_test_pred = model.predict(X_test)

        # Train metrics
        train_accuracy = model.score(X_train, y_train)
        train_precision = precision_score(y_train, y_train_pred,
↪average="binary")
        train_recall = recall_score(y_train, y_train_pred, average="binary")
        train_f1 = f1_score(y_train, y_train_pred, average="binary")

        # Test metrics
        test_accuracy = model.score(X_test, y_test)
        test_precision = precision_score(y_test, y_test_pred, average="binary")
        test_recall = recall_score(y_test, y_test_pred, average="binary")
        test_f1 = f1_score(y_test, y_test_pred, average="binary")

        # Append results
        results.append({
            "Model": name,
            "Train Accuracy": train_accuracy,
            "Test Accuracy": test_accuracy,
            "Train Precision": train_precision,
            "Test Precision": test_precision,
            "Train Recall": train_recall,
            "Test Recall": test_recall,
            "Train F1-Score": train_f1,
            "Test F1-Score": test_f1,
        })
    except Exception as e:
        print(f"Could not process {name}: {e}")
        results.append({
            "Model": name,
            "Train Accuracy": np.nan,
            "Test Accuracy": np.nan,
            "Train Precision": np.nan,
            "Test Precision": np.nan,
            "Train Recall": np.nan,

```

```

        "Test Recall": np.nan,
        "Train F1-Score": np.nan,
        "Test F1-Score": np.nan,
    })

```

/Users/sriharsha/Desktop/anaconda3/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero\_division` parameter to control this behavior.

```

_warn_prf(average, modifier, msg_start, len(result))

```

```

[14]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Convert results to a DataFrame
results_df = pd.DataFrame(results)
results_df.to_csv("classification_train_test_metrics.csv", index=False)
print("\nMetrics saved to 'classification_train_test_metrics.csv'.")

# Plot bar charts for each metric
metrics = ["Accuracy", "Precision", "Recall", "F1-Score"]
for metric in metrics:
    train_values = results_df[f"Train {metric}"]
    test_values = results_df[f"Test {metric}"]
    x = np.arange(len(results_df["Model"])) # Label locations
    width = 0.35 # Bar width

    fig, ax = plt.subplots(figsize=(12, 6))

    # Bar plots
    bars1 = ax.bar(x - width/2, train_values, width, label=f"Train {metric}",
        color="blue", alpha=0.7)
    bars2 = ax.bar(x + width/2, test_values, width, label=f"Test {metric}",
        color="orange", alpha=0.7)

    # Add labels, title, and legend
    ax.set_xlabel("Models")
    ax.set_ylabel(metric)
    ax.set_title(f"Train vs Test {metric} Comparison Across Models")
    ax.set_xticks(x)
    ax.set_xticklabels(results_df["Model"], rotation=45, ha="right")
    ax.legend()

    # Annotate bars with values
    for bar in bars1 + bars2:
        height = bar.get_height()

```

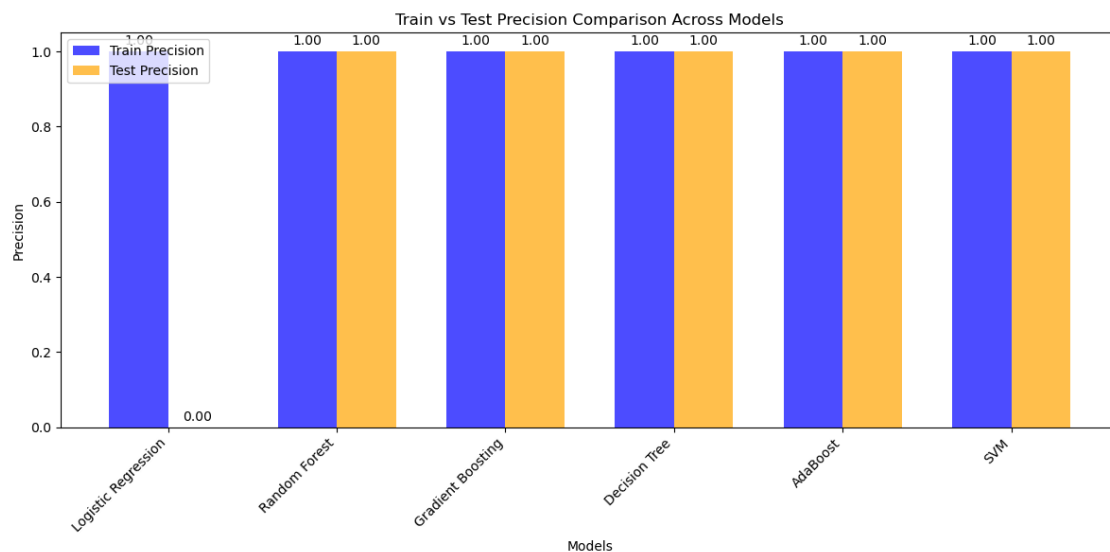
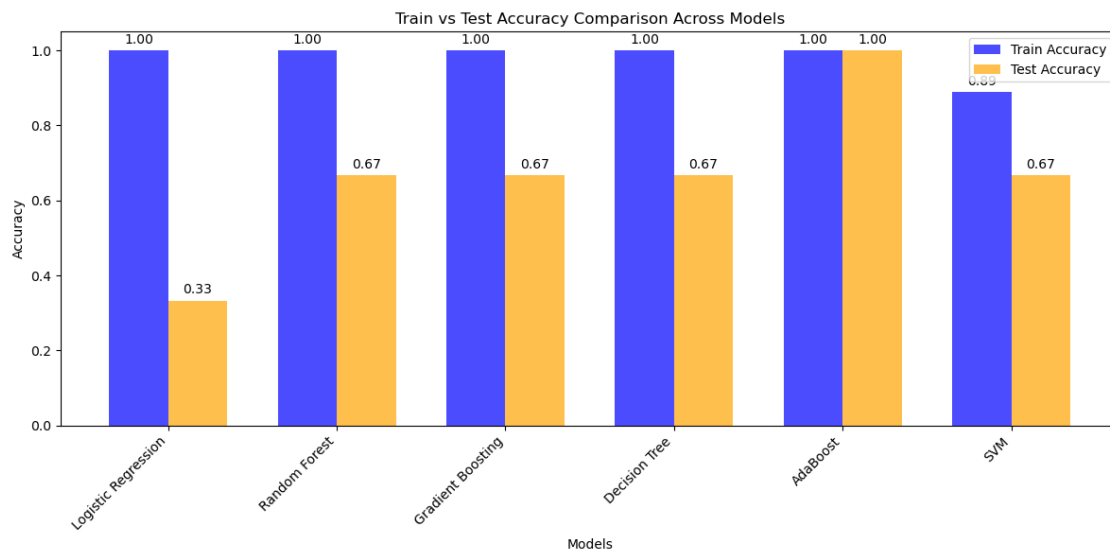
```

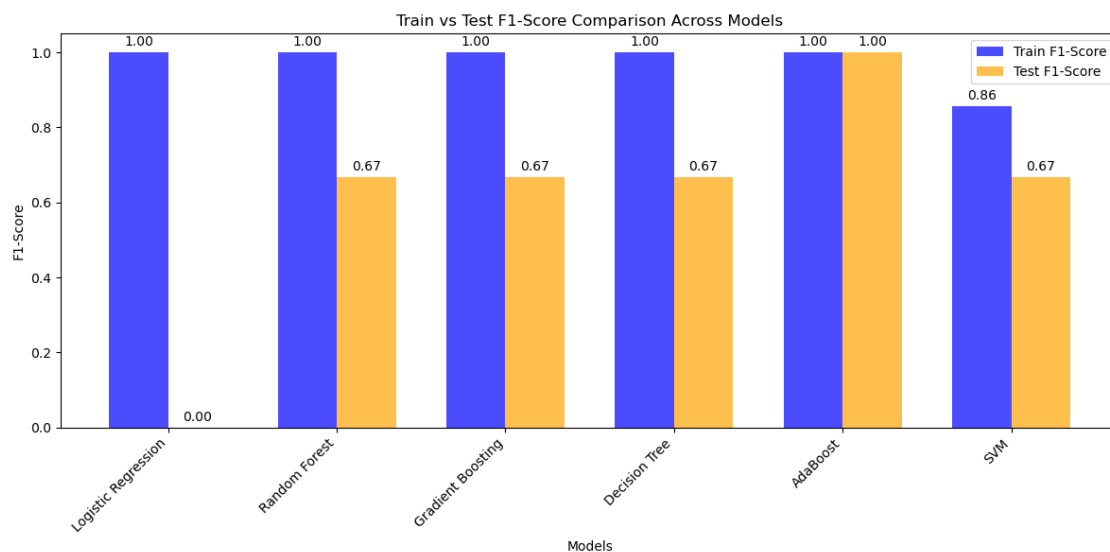
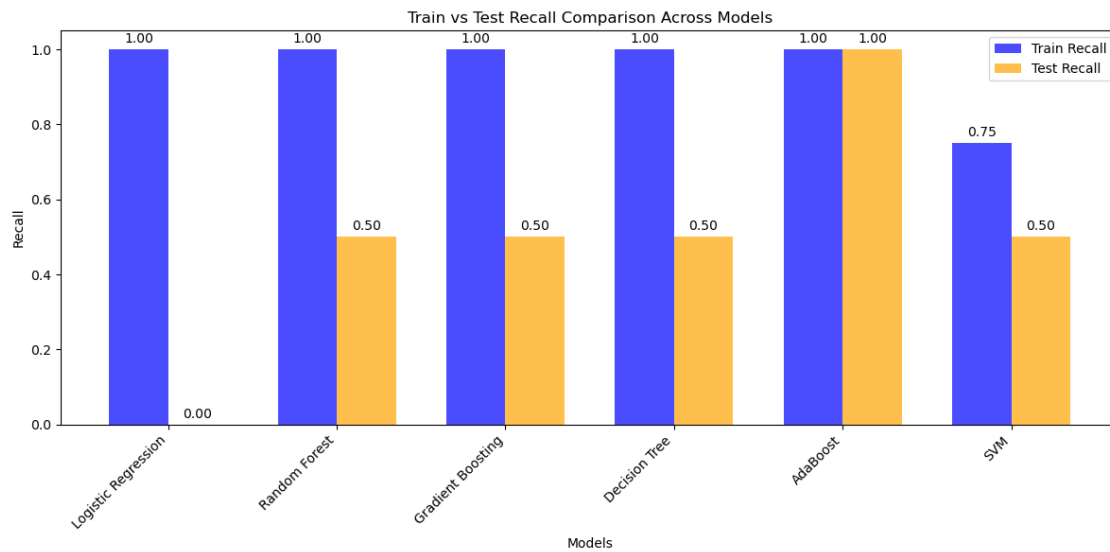
ax.annotate(f"{height:.2f}",
            xy=(bar.get_x() + bar.get_width() / 2, height),
            xytext=(0, 3), # Offset text by 3 points
            textcoords="offset points",
            ha="center", va="bottom")

# Display and save the plot
plt.tight_layout()
plt.savefig(f"train_test_{metric.lower()}_comparison.png")
plt.show()

```

Metrics saved to 'classification_train_test_metrics.csv'.





```
[15]: # Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
```

```

from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import GridSearchCV
import matplotlib.pyplot as plt
import seaborn as sns

# Load and preprocess the data
data = pd.read_csv("DataSet.csv")

```

```

[16]: data.rename(columns={
        'Asian/Asian British': 'Asian',
        'Black/African/Caribbean/Black British': 'Black',
        'Mixed/Multiple ethnic groups': 'Mixed',
        'Not Stated': 'Not_Stated',
        'Other ethnic group': 'Other',
        'Clinical condition group': 'Condition_Group'
    }, inplace=True)

[17]: # Create binary target variable for regression-like High Risk
data['High_Risk'] = (data['Total'] > data['Total'].median()).astype(int)

# Feature Engineering
X_features = data.drop(columns=['Condition_Group', 'Total', 'High_Risk',
    ↪ 'Female', 'Male',
    ↪ '12 to 16', '17 to 19', '20 to 29', '30 to 39',
    ↪ '40 to 49',
    ↪ '50 to 59', '60 to 69', '70 to 79', '80 to 89',
    ↪ '90+'])
X = X_features
y = data['High_Risk']

[18]: # Handle class imbalance using SMOTE
smote = SMOTE(random_state=42, k_neighbors=1)
X_balanced, y_balanced = smote.fit_resample(X, y)

[19]: # Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_balanced, y_balanced,
    ↪ test_size=0.2, random_state=42)

[20]: # Define models for regression-like High Risk
regression_models = {
    "Linear Regression": LinearRegression(),
    "Random Forest Regressor": RandomForestRegressor(random_state=42),
    "Gradient Boosting Regressor": GradientBoostingRegressor(random_state=42),
    "Decision Tree Regressor": DecisionTreeRegressor(random_state=42)
}

```



```
[21]: # Define hyperparameter grids for regression models
param_grids = {
    "Random Forest Regressor": {
        'n_estimators': [100, 200],
        'max_depth': [10, 20]
    },
    "Gradient Boosting Regressor": {
        'n_estimators': [100, 200],
        'learning_rate': [0.01, 0.1]
    },
    "Decision Tree Regressor": {
        'max_depth': [10, 20],
        'min_samples_split': [2, 5]
    }
}

[22]: import pandas as pd
import numpy as np
from sklearn.metrics import mean_squared_error, r2_score

# Initialize a list to store evaluation results for each regression model
regression_results = []

# Iterate through the regression models
for name, model in regression_models.items():
    print(f"Evaluating {name}...") # Log the model being evaluated

    # Train the model using the training data
    model.fit(X_train, y_train)

    # Generate predictions on the test data
    y_pred = model.predict(X_test)

    # Calculate regression evaluation metrics
    mse = mean_squared_error(y_test, y_pred) # Mean Squared Error
    rmse = np.sqrt(mse) # Root Mean Squared Error
    r2 = r2_score(y_test, y_pred) # R-squared score

    # Append the evaluation metrics for the current model to the results list
    regression_results.append({
        "Model": name, # Model name
        "MSE": mse, # Mean Squared Error
        "RMSE": rmse, # Root Mean Squared Error
        "R2": r2 # R-squared score
    })
```

```

# Convert the results to a DataFrame
results_df = pd.DataFrame(regression_results)

# Save the results to a CSV file
results_df.to_csv("regression_results.csv", index=False)

# Print confirmation message
print("Regression evaluation results have been saved to 'regression_results.
↵csv'.")

```

Evaluating Linear Regression...

Evaluating Random Forest Regressor...

Evaluating Gradient Boosting Regressor...

Evaluating Decision Tree Regressor...

Regression evaluation results have been saved to 'regression_results.csv'.

```

[23]: import matplotlib.pyplot as plt
import seaborn as sns

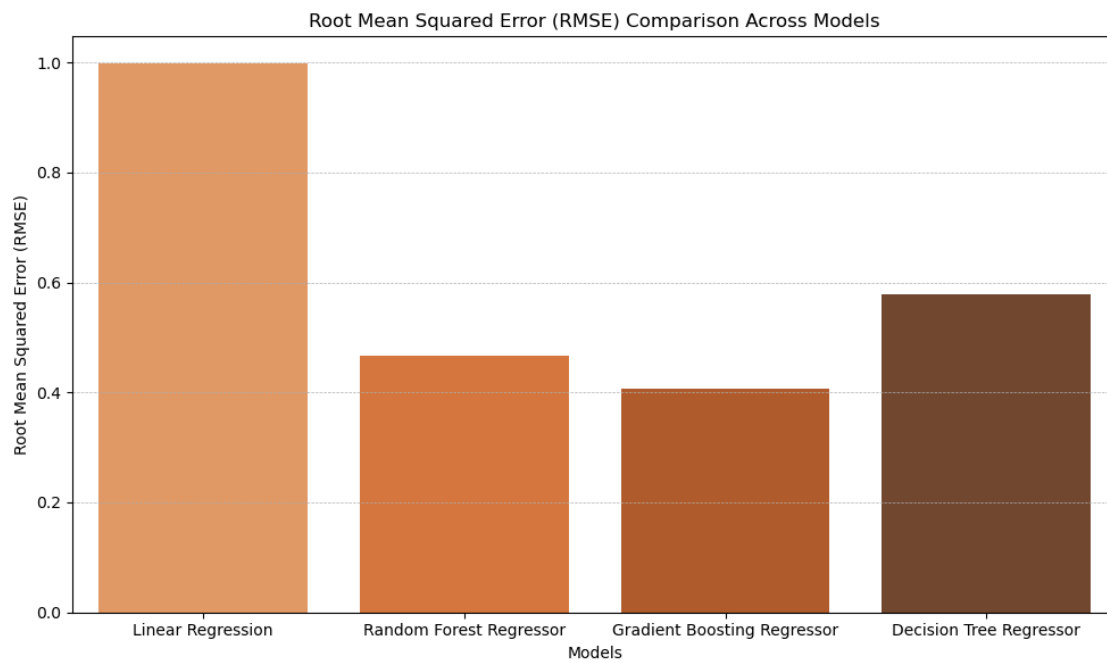
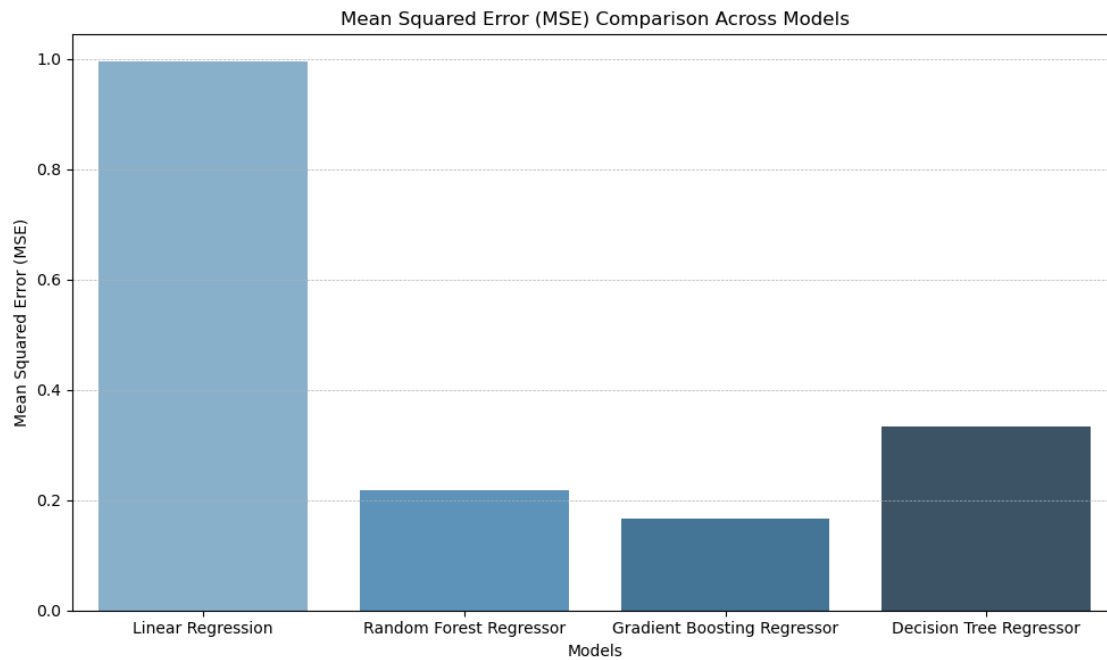
# Plot Mean Squared Error (MSE)
plt.figure(figsize=(10, 6))
sns.barplot(data=results_df, x="Model", y="MSE", palette="Blues_d")
plt.title("Mean Squared Error (MSE) Comparison Across Models")
plt.ylabel("Mean Squared Error (MSE)")
plt.xlabel("Models")
plt.grid(axis="y", linestyle="--", linewidth=0.5)
plt.tight_layout()
plt.savefig("mse_comparison.png")
plt.show()

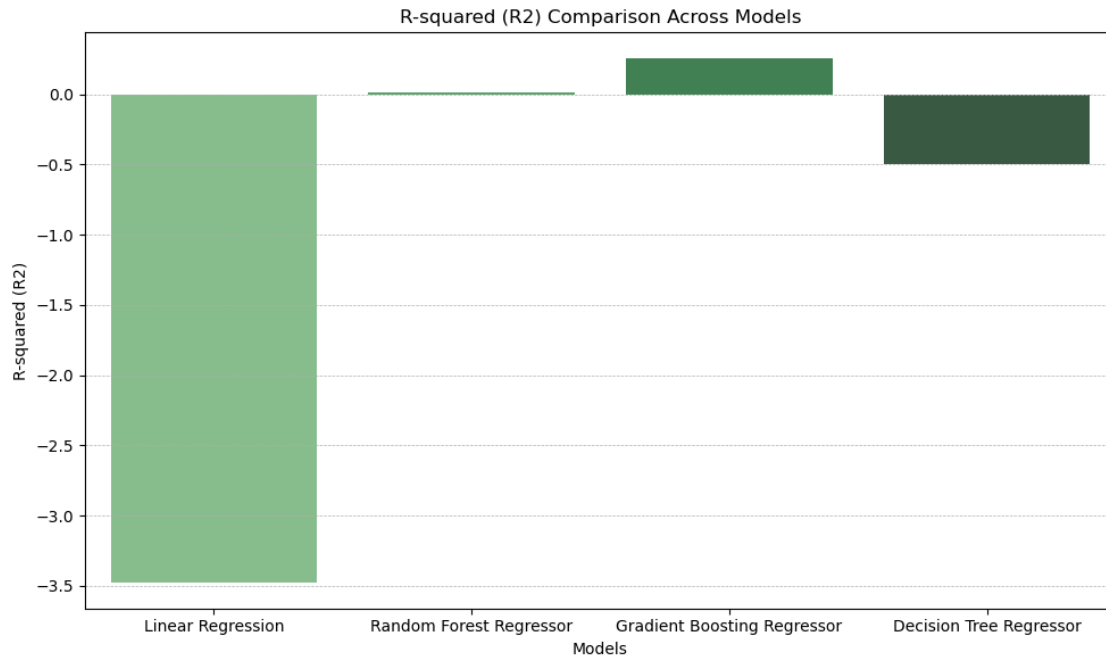
# Plot Root Mean Squared Error (RMSE)
plt.figure(figsize=(10, 6))
sns.barplot(data=results_df, x="Model", y="RMSE", palette="Oranges_d")
plt.title("Root Mean Squared Error (RMSE) Comparison Across Models")
plt.ylabel("Root Mean Squared Error (RMSE)")
plt.xlabel("Models")
plt.grid(axis="y", linestyle="--", linewidth=0.5)
plt.tight_layout()
plt.savefig("rmse_comparison.png")
plt.show()

# Plot R-squared (R2)
plt.figure(figsize=(10, 6))
sns.barplot(data=results_df, x="Model", y="R2", palette="Greens_d")
plt.title("R-squared (R2) Comparison Across Models")
plt.ylabel("R-squared (R2)")
plt.xlabel("Models")

```

```
plt.grid(axis="y", linestyle="--", linewidth=0.5)
plt.tight_layout()
plt.savefig("r2_comparison.png")
plt.show()
```





```
[24]: # Calculate metrics for Train and Test datasets
results = []

for name, model in regression_models.items():
    try:
        # Fit the model
        model.fit(X_train, y_train)

        # Train predictions
        y_train_pred = model.predict(X_train)
        # Test predictions
        y_test_pred = model.predict(X_test)

        # Train metrics
        train_mse = mean_squared_error(y_train, y_train_pred)
        train_rmse = np.sqrt(train_mse)
        train_r2 = r2_score(y_train, y_train_pred)

        # Test metrics
        test_mse = mean_squared_error(y_test, y_test_pred)
        test_rmse = np.sqrt(test_mse)
        test_r2 = r2_score(y_test, y_test_pred)

        # Append results
        results.append({
```

```

        "Model": name,
        "Train MSE": train_mse,
        "Test MSE": test_mse,
        "Train RMSE": train_rmse,
        "Test RMSE": test_rmse,
        "Train R2": train_r2,
        "Test R2": test_r2,
    })
except Exception as e:
    print(f"Could not process {name}: {e}")
    results.append({
        "Model": name,
        "Train MSE": np.nan,
        "Test MSE": np.nan,
        "Train RMSE": np.nan,
        "Test RMSE": np.nan,
        "Train R2": np.nan,
        "Test R2": np.nan,
    })

# Convert results to a DataFrame
results_df = pd.DataFrame(results)
results_df.to_csv("regression_train_test_metrics.csv", index=False)
print("\nMetrics saved to 'regression_train_test_metrics.csv'.")

```

Metrics saved to 'regression_train_test_metrics.csv'.

```

[25]: # Plot bar charts for each metric
metrics = ["MSE", "RMSE", "R2"]
for metric in metrics:
    train_values = results_df[f"Train {metric}"]
    test_values = results_df[f"Test {metric}"]
    x = np.arange(len(results_df["Model"])) # Label locations
    width = 0.35 # Bar width

    fig, ax = plt.subplots(figsize=(12, 6))

    # Bar plots
    bars1 = ax.bar(x - width / 2, train_values, width, label=f"Train {metric}",
        color="blue", alpha=0.7)
    bars2 = ax.bar(x + width / 2, test_values, width, label=f"Test {metric}",
        color="orange", alpha=0.7)

    # Add labels, title, and legend
    ax.set_xlabel("Models")
    ax.set_ylabel(metric)

```

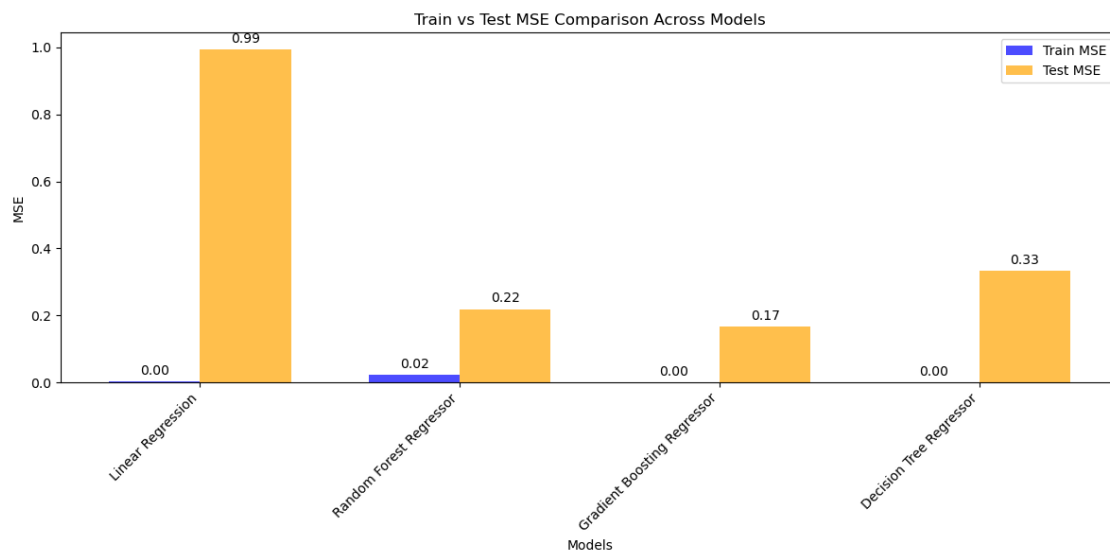
```

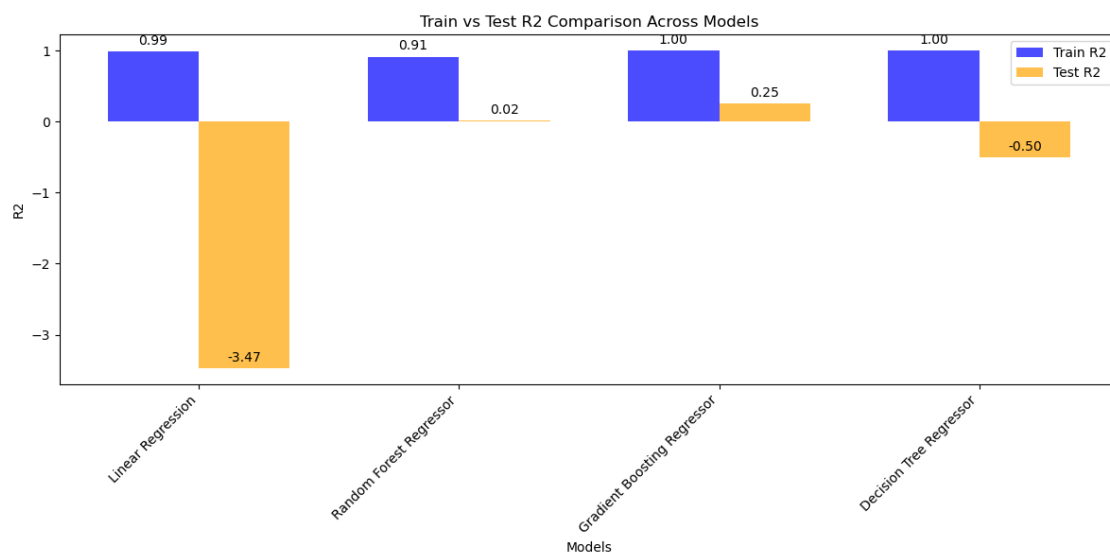
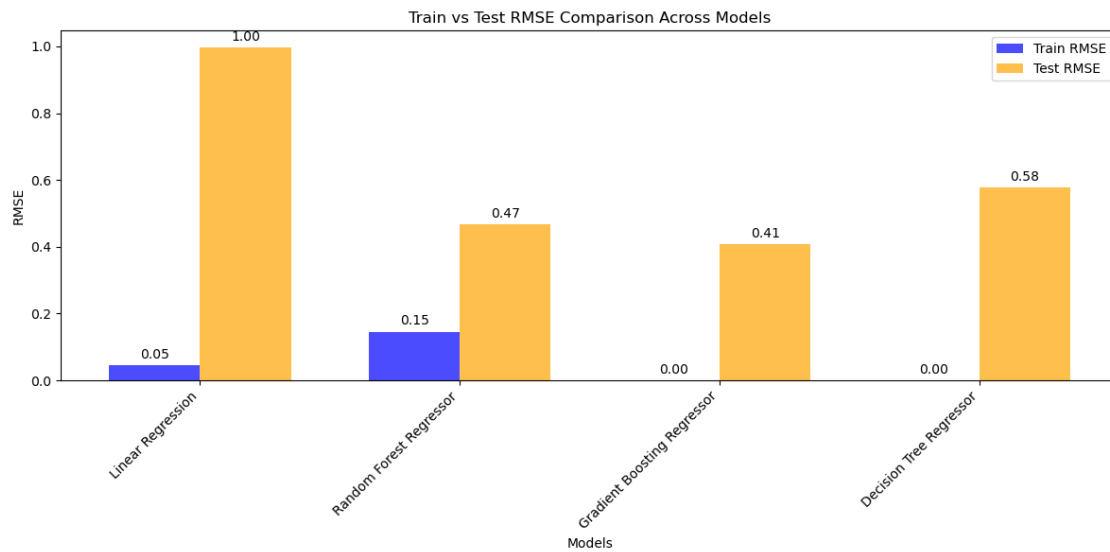
ax.set_title(f"Train vs Test {metric} Comparison Across Models")
ax.set_xticks(x)
ax.set_xticklabels(results_df["Model"], rotation=45, ha="right")
ax.legend()

# Annotate bars with values
for bar in bars1 + bars2:
    height = bar.get_height()
    ax.annotate(f"{height:.2f}",
                xy=(bar.get_x() + bar.get_width() / 2, height),
                xytext=(0, 3), # Offset text by 3 points
                textcoords="offset points",
                ha="center", va="bottom")

# Display and save the plot
plt.tight_layout()
plt.savefig(f"train_test_{metric.lower()}_comparison.png")
plt.show()

```





[]: